LAPORAN PRAKTIKUM ALGORITMA DAN STRUKTUR DATA

OLEH:

Nama	: Aldho Perayudha
Nim	: 2025573010117
Kelas	: TI 1\C
Jurusan	: Teknologi Informasi dan Komputer
Dosen	: Muhammad.Reza Zulman, S.ST., M.Sc

PROGRAM STUDI TEKNIK INFORMATIKA
JURUSAN TEKNOLOGI INFORMASI DAN KOMPUTER
POLITEKNIK NEGERI LHOKSEUMAWE
2026

LEMBAR PENGESAHAN

Nama Mata Kuliah	: Pratikum Algoritma dan struktur data
Judul Pratikum	: Praktikum Analisis Kompleksitas Algoritma
Hari/Tanggal Pratikum	: 22 April 2026
Tanggal Penyerahan	: 28 April 2026
Tempat	: Laboratorium (TIK.106)
Nama Mahasiswa	: Aldho Perayudha
NIM	: 2025573010117
Program Studi/Kelas	: Teknik Informatika / TI.1C

Mengetahui
Dosen Pengajar,

Muhammad.Reza Zulman, S.ST., M.Sc

KATA PENGANTAR

Puji syukur penulis ucapkan ke hadirat Allah SWT atas rahmat dan karunianya sehingga laporan praktikum yang berjudul "Analisis Kompleksitas Algoritma (Big O Notation, Time & Space Complexity)" ini dapat diselesaikan dengan baik. Laporan ini disusun sebagai bentuk tanggung jawab atas kegiatan praktikum yang telah dilakukan, sekaligus untuk memenuhi tugas yang diberikan.

Penyusunan laporan ini bertujuan untuk memahami dasar-dasar analisis algoritma dalam pemrograman, khususnya dengan menggunakan JavaScript dan Node.js. Melalui praktikum ini, penulis mencoba memahami bagaimana cara mengukur efisiensi algoritma menggunakan Big O Notation, serta membedakan antara kompleksitas waktu (time complexity) dan kompleksitas ruang (space complexity).

Selain itu, penulis juga mempelajari bagaimana ukuran input dapat memengaruhi kinerja suatu program. Dari sini, penulis mulai memahami pentingnya memilih algoritma yang tepat agar solusi yang dihasilkan lebih efisien dan tidak memakan banyak sumber daya.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, penulis sangat terbuka terhadap kritik dan saran yang membangun agar ke depannya dapat menjadi lebih baik. Semoga laporan ini dapat bermanfaat dan menambah wawasan, khususnya di bidang struktur data dan algoritma.

BAB I
PENDAHULUAN

1. Latar Belakang

Seiring berkembangnya teknologi, kemampuan dalam membuat algoritma yang efisien menjadi hal yang penting dalam dunia pemrograman. Tidak hanya sekadar membuat program berjalan, tetapi juga perlu diperhatikan bagaimana program tersebut dapat berjalan dengan cepat dan menggunakan sumber daya yang optimal.

Dalam hal ini, analisis kompleksitas algoritma menjadi salah satu aspek penting yang harus dipahami. Big O Notation digunakan sebagai alat untuk mengukur tingkat efisiensi suatu algoritma, baik dari segi waktu eksekusi maupun penggunaan memori.

Dengan menggunakan JavaScript dan Node.js, analisis ini dapat dilakukan melalui berbagai percobaan sederhana untuk melihat bagaimana algoritma bekerja pada berbagai ukuran data. Oleh karena itu, praktikum ini bertujuan untuk memberikan pemahaman langsung terkait konsep tersebut.

2. **Tujuan Praktikum**

   Adapun tujuan dari praktikum ini adalah:

   - Memahami konsep dasar Big O Notation
   - Mengetahui perbedaan time complexity dan space complexity
   - Menganalisis performa algoritma berdasarkan ukuran input
   - Mampu mengimplementasikan dan menguji kompleksitas algoritma menggunakan JavaScript

BAB II
LANDASAN TEORI

1. **Pengertian Algoritma**

   Algoritma adalah langkah-langkah sistematis yang digunakan untuk menyelesaikan suatu masalah. Dalam pemrograman, algoritma menjadi dasar dalam pembuatan program yang efisien.

2. **Big O Notation**

   Big O Notation merupakan notasi matematis yang digunakan untuk menggambarkan tingkat pertumbuhan suatu algoritma terhadap ukuran input. Notasi ini membantu dalam membandingkan efisiensi beberapa algoritma.

3. **Time Complexity**

   Time complexity adalah ukuran yang menunjukkan seberapa lama waktu yang dibutuhkan suatu algoritma untuk dijalankan berdasarkan jumlah data yang diproses**.**

4. **Space Complexity**

   Space complexity mengukur jumlah memori yang digunakan oleh suatu algoritma selama proses eksekusi berlangsung.

5. **Jenis-Jenis Kompleksitas**

   Beberapa jenis kompleksitas yang digunakan antara lain :

   - O(1) → Konstan
   - O(n) → Linear

- $O(n^2)$ → Kuadratik
- $O(\log n)$ → Logaritmik

BAB III
METODOLOGI PRAKTIKUM

1. Alat dan Bahan

1. Pc/Laptop
2. Koneksi internet
3. Visual Studio Code (VSCode)
4. Gitbush
5. Terminal

2. Langkah-langkah Praktikum

1. Buka terminal, navigasikan ke root folder repository kamu. Buat folder praktikum-5/ dan masuk ke dalamnya. Ketik “mkdir praktikum-5” Setelah itu dilanjutkan ketik “cd praktikum-5”. Inisialisasi npm project “npm init -y”.

```
HP@LAPTOP-PK3JNVV7 MINGW64 /c/2025573010117_Praktikum-Algoritma_dan-Struktur_data (main)
$ mkdir praktikum-5

HP@LAPTOP-PK3JNVV7 MINGW64 /c/2025573010117_Praktikum-Algoritma_dan-Struktur_data (main)
$ cd praktikum-5

HP@LAPTOP-PK3JNVV7 MINGW64 /c/2025573010117_Praktikum-Algoritma_dan-Struktur_data/praktikum-5 (main)
$ npm init -y
Wrote to C:\2025573010117_Praktikum-Algoritma_dan-Struktur_data\praktikum-5\package.json:

{
  "name": "praktikum-5",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

2. Kemudian lanjut Buat file baru bernama 01-intro- kompleksitas.js. Ketik “touch 01-intro-kompleksitas.js” dan Pastikan kamu berada di dalam folder praktikum-5/.

```
HP@LAPTOP-PK3JNVV7 MINGW64 /c/2025573010117_Praktikum-Algoritma_dan-Struktur_data/praktikum-5 (main)
$ touch 01-intro-kompleksitas.js
```

3. Lanjut Buka file 01-intro-kompleksitas.js di VS Code, lalu ketikkan kode berikut.

```javascript
// 01-intro-kompleksitas.js
// Mengukur perbedaan performa algoritma secara empiris

// Fungsi bantu: ukur waktu eksekusi
function ukurWaktu(label, fn) {
  const mulai = Date.now();
  fn();
  const selesai = Date.now();
  console.log(`${label}: ${selesai - mulai} ms`);
}
const N = 100_000; // ukuran data: 100 ribu

// --- Algoritma A: O(n) — loop satu kali ---
function jumlahkanLinear(n) {
  let total = 0;
  for (let i = 1; i <= n; i++) total += i;
  return total;
}

// --- Algoritma B: O(1) — rumus matematika ---
function jumlahkanRumus(n) {
  return (n * (n + 1)) / 2;
}

// --- Algoritma C: O(n2) — loop bersarang ---
function cariPasangan(arr) {
  const pasangan = [];
  for (let i = 0; i < arr.length; i++) {
    for (let j = i + 1; j < arr.length; j++) {
      if (arr[i] + arr[j] === 10) pasangan.push([arr[i], arr[j]]);
    }
  }
  return pasangan;
}

const data = Array.from({ length: 5000 }, (_, i) => i);

console.log("=== Perbandingan Waktu Eksekusi ===");
ukurWaktu("O(1) jumlahkanRumus ", () => jumlahkanRumus(N));
ukurWaktu("O(n) jumlahkanLinear ", () => jumlahkanLinear(N));
ukurWaktu("O(n2) cariPasangan ", () => cariPasangan(data));

console.log("\nHasil sama?", jumlahkanLinear(100) === jumlahkanRumus(100));
```

4. Kemudian Simpan file, lalu jalankan dari terminal. Ketik “node 01-intro-kompleksitas.js”.

```
HP@LAPTOP-PK3JNVV7 MINGW64 /c/2025573010117_Praktikum-Algoritma_dan-Struktur_data/praktikum-5 (main)
$ node 01-intro-kompleksitas.js
=== Perbandingan Waktu Eksekusi ===
O(1) jumlahkanRumus : 0 ms
O(n) jumlahkanLinear : 2 ms
O(n2) cariPasangan : 28 ms

Hasil sama? true
```

5. Lanjut Buat file baru bernama 02-kelas-big-o.js di dalam folder praktikum-5/. Ketik “touch 02-destructuring-spread.js”.

```
HP@LAPTOP-PK3JNVV7 MINGW64 /c/2025573010117_Praktikum-Algoritma_dan-Struktur_data/praktikum-5 (main)
$ touch 02-kelas-big-o.js
```

6. Buka file 02-kelas-big-o.js di VS Code, lalu ketikkan kode berikut.

```javascript
// 02-kelas-big-o.js
// Contoh konkret setiap kelas kompleksitas Big O

// ── O(1) — Konstan ──────────────────────────────
function ambilPertama(arr) {
  return arr[0];
} // O(1)
function tambahkanItem(arr, item) {
  arr.push(item);
} // O(1)
function isGenap(n) {
  return n % 2 === 0;
} // O(1)

// ── O(log n) — Logaritmik ───────────────────────
function binarySearch(arr, target) {
  let kiri = 0,
    kanan = arr.length - 1;
  let langkah = 0;
  while (kiri <= kanan) {
    langkah++;
    const tengah = Math.floor((kiri + kanan) / 2);
    if (arr[tengah] === target) {
      console.log(` Ditemukan di indeks ${tengah} setelah ${langkah} langkah`);
      return tengah;
    }
    if (arr[tengah] < target) kiri = tengah + 1;
    else kanan = tengah - 1;
  }
  return -1;
}

// ── O(n) — Linear ───────────────────────────────
function cariMax(arr) {
  let maks = arr[0];
  for (const x of arr) if (x > maks) maks = x;
  return maks;
}

// ── O(n2) — Kuadratik ───────────────────────────
function bubbleSortNaif(arr) {
  const a = [...arr];
  for (let i = 0; i < a.length; i++)
    for (let j = 0; j < a.length - i - 1; j++)
      if (a[j] > a[j + 1]) [a[j], a[j + 1]] = [a[j + 1], a[j]];
  return a;
}

// ── O(2n) — Eksponensial ────────────────────────
function fibRekursif(n) {
  if (n <= 1) return n;
  return fibRekursif(n - 1) + fibRekursif(n - 2); // dua panggilan rekursif!
}

// ── Demonstrasi ─────────────────────────────────
console.log("=== O(1) — selalu cepat ===");
console.log(ambilPertama([10, 20, 30, 40, 50])); // 10
console.log(isGenap(42)); // true

console.log("\n=== O(log n) — Binary Search ===");
const sorted = Array.from({ length: 1_000_000 }, (_, i) => i);
binarySearch(sorted, 731_452); // cari di 1 juta data

console.log("\n=== O(n) — Linear Search ===");
console.log(
  "Max dari 1000 elemen:",
  cariMax(Array.from({ length: 1000 }, () => (Math.random() * 1000) | 0)),
);

console.log("\n=== O(n2) — Bubble Sort (kecil saja) ===");
console.log(bubbleSortNaif([64, 34, 25, 12, 22, 11, 90]));

console.log("\n=== O(2n) — Fibonacci Rekursif (n kecil!) ===");
for (let i = 0; i <= 10; i++) process.stdout.write(fibRekursif(i) + " ");
console.log("");

// Demonstrasi betapa lambatnya O(2n)
console.log("\nWaktu fib(35) O(2n):");
let t = Date.now();
fibRekursif(35);
console.log(Date.now() - t, "ms");

console.log("Waktu fib(35) O(n) memoization:");
const memo = {};
function fibMemo(n) {
  if (n <= 1) return n;
  if (memo[n]) return memo[n];
  return (memo[n] = fibMemo(n - 1) + fibMemo(n - 2));
}
t = Date.now();
fibMemo(35);
console.log(Date.now() - t, "ms");
```

7. Simpan file, lalu jalankan dari terminal. Dengan cara ketik “node 02-kelas-big-o.js”.

```
HP@LAPTOP-PK3JNVV7 MINGW64 /c/2025573010117_Praktikum-Algoritma_dan-Struktur_data/praktikum-5 (main)
$ node 02-kelas-big-o.js
=== O(1) — selalu cepat ===
10
true

=== O(log n) — Binary Search ===
 Ditemukan di indeks 731452 setelah 19 langkah

=== O(n) — Linear Search ===
Max dari 1000 elemen: 998

=== O(n2) — Bubble Sort (kecil saja) ===
[
  11, 12, 22, 25,
  34, 64, 90
]

=== O(2n) — Fibonacci Rekursif (n kecil!) ===
0 1 1 2 3 5 8 13 21 34 55

Waktu fib(35) O(2n):
171 ms
Waktu fib(35) O(n) memoization:
1 ms
```

8. Buat kembali file baru bernama 03-space-complexity.js. ketik “touch 03-space-complexity.js”.

```
HP@LAPTOP-PK3JNVV7 MINGW64 /c/2025573010117_Praktikum-Algoritma_dan-Struktur_data/praktikum-5 (main)
$ touch 03-space-complexity.js
```

9. Buka file 03-space-complexity.js di VS Code, Kemudian lanjut ketikkan kode berikut, Beserta dengan kode latihan2.

```javascript
// 03-space-complexity.js

// --- O(1) Space: hanya pakai variabel tambahan konstan ---
function jumlahArray(arr) {
  let total = 0; // hanya 1 variabel tambahan
  for (const x of arr) total += x;
  return total;
}

// --- O(n) Space: membuat struktur baru sebesar input ---
function duplikasiArray(arr) {
  const baru = []; // array baru tumbuh seiring arr
  for (const x of arr) baru.push(x * 2);
  return baru;
}

// --- O(n) Space: rekursi (call stack) ---
function faktorialRekursif(n) {
  if (n <= 1) return 1;
  return n * faktorialRekursif(n - 1); // n frame di call stack
}

// --- O(1) Space: faktorial iteratif ---
function faktorialIteratif(n) {
  let hasil = 1;
  for (let i = 2; i <= n; i++) hasil *= i; // hanya 2 variabel
  return hasil;
}

const arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];

console.log("Jumlah array :", jumlahArray(arr)); // O(1) space
console.log("Duplikasi array:", duplikasiArray(arr)); // O(n) space
console.log("Faktorial 10 rekursif :", faktorialRekursif(10));
console.log("Faktorial 10 iteratif :", faktorialIteratif(10));

// Visualisasi: hitung elemen unik (O(n) space)
function hitungUnik(arr) {
  const seen = new Set(); // Set tumbuh hingga n elemen
  for (const x of arr) seen.add(x);
  return seen.size;
}

const dataAcak = [1, 2, 3, 2, 1, 4, 5, 3, 6, 4, 7];
console.log("Elemen unik:", hitungUnik(dataAcak)); // 7

// ==========================
// LATIHAN 3 - PAIR SUM
// ==========================

// Fungsi Lambat (Brute Force)
function cariPasanganLambat(arr, target) {
  for (let i = 0; i < arr.length; i++) {
    for (let j = i + 1; j < arr.length; j++) {
      if (arr[i] + arr[j] === target) {
        return [arr[i], arr[j]];
      }
    }
  }
  return null;
}
// Big O Time: O(n^2)
// Alasan: nested loop (dua perulangan)
// Big O Space: O(1)
// Alasan: tidak menggunakan struktur data tambahan

// Fungsi Cepat (Menggunakan Set)
function cariPasanganCepat(arr, target) {
  const seen = new Set();

  for (let num of arr) {
    const complement = target - num;
    if (seen.has(complement)) {
      return [complement, num];
    }
    seen.add(num);
  }

  return null;
}
// Big O Time: O(n)
// Alasan: hanya satu loop
// Big O Space: O(n)
// Alasan: menggunakan Set untuk menyimpan data

// ==========================
// UJI COBA SEDERHANA
// ==========================
const arrTest = [2, 7, 11, 15];
const targetTest = 9;

console.log("Lambat:", cariPasanganLambat(arrTest, targetTest)); // [2,7]
console.log("Cepat:", cariPasanganCepat(arrTest, targetTest)); // [2,7]

// ==========================
// TEST PERFORMA (50.000 DATA)
// ==========================

// buat array random
function generateArray(n) {
  return Array.from({ length: n }, () => Math.floor(Math.random() * 100000));
}

const bigArray = generateArray(50000);
const target = -1; // sengaja tidak ada

console.log("\n=== TEST PERFORMA ===");

// Lambat
let start = Date.now();
cariPasanganLambat(bigArray, target);
let end = Date.now();
console.log("Lambat:", end - start, "ms");

// Cepat
start = Date.now();
cariPasanganCepat(bigArray, target);
end = Date.now();
console.log("Cepat:", end - start, "ms");
```

10. Simpan file, dan jalankan dari terminal, Ketik “node 03-space-complexity.js” amati hasilnya.

```
HP@LAPTOP-PK3JNVV7 MINGW64 /c/2025573010117_Praktikum-Algoritma_dan-Struktur_data/praktikum-5 (main)
$ node 03-space-complexity.js
Jumlah array : 55
Duplikasi array: [
   2,  4,  6,  8, 10,
  12, 14, 16, 18, 20
]
Faktorial 10 rekursif : 3628800
Faktorial 10 iteratif : 3628800
Elemen unik: 7
```

11. Kemudian Buat kembali file baru bernama 04-refactor.js di dalam folder praktikum-5/. Ketik “touch 04-refactor.js”.

```
HP@LAPTOP-PK3JNVV7 MINGW64 /c/2025573010117_Praktikum-Algoritma_dan-Struktur_data/praktikum-5 (main)
$ touch 04-refactor.js
```

12. Lanjut Buka file 04-refactor.js di VS Code, lalu ketikkan kode berikut.

```javascript
// 05-refactor.js — Refactoring kode berperforma buruk
// ═══════════════════════════════════════════════
// SKENARIO 1: Cek duplikat dalam array
// ═══════════════════════════════════════════════

// BURUK: O(n2) — nested loop
function adaDuplikatLambat(arr) {
  for (let i = 0; i < arr.length; i++)
    for (let j = i + 1; j < arr.length; j++) if (arr[i] === arr[j]) return true;
  return false;
}

// BAIK: O(n) — gunakan Set
function adaDuplikatCepat(arr) {
  const seen = new Set();
  for (const x of arr) {
    if (seen.has(x)) return true;
    seen.add(x);
  }
  return false;
}

// ═══════════════════════════════════════════════
// SKENARIO 2: Frekuensi kemunculan elemen
// ═══════════════════════════════════════════════
// BURUK: O(n2) — indexOf di dalam loop
function frekuensiLambat(arr) {
  const unik = [];
  const hitung = [];
  for (const x of arr) {
    const idx = unik.indexOf(x); // O(n) di dalam loop O(n)
    if (idx === -1) {
      unik.push(x);
      hitung.push(1);
    } else hitung[idx]++;
  }
  return Object.fromEntries(unik.map((u, i) => [u, hitung[i]]));
}

// BAIK: O(n) — gunakan object/Map sebagai counter
function frekuensiCepat(arr) {
  const counter = {};
  for (const x of arr) counter[x] = (counter[x] || 0) + 1;
  return counter;
}

// ── Benchmark ──────────────────────────────
const besar = Array.from({ length: 20000 }, () =>
  Math.floor(Math.random() * 1000),
);

let t = Date.now();
adaDuplikatLambat(besar);
console.log("Duplikat LAMBAT O(n2):", Date.now() - t, "ms");

t = Date.now();
adaDuplikatCepat(besar);
console.log("Duplikat CEPAT O(n) :", Date.now() - t, "ms");

const kata = ["a", "b", "a", "c", "b", "a", "d"];
console.log("\nFrekuensi:", frekuensiCepat(kata));
```

13. Simpan file, dan jalankan dari terminal. Lalu amati hasilnya dengan cara ketik “node 04-refactor.js”.

```
HP@LAPTOP-PK3JNVV7 MINGW64 /c/2025573010117_Praktikum-Algoritma_dan-Struktur_data/praktikum-5 (main)
$ node 04-refactor.js
Duplikat LAMBAT O(n2): 1 ms
Duplikat CEPAT O(n) : 0 ms

Frekuensi: { a: 3, b: 2, c: 1, d: 1 }
```

14. Lanjut Buat kembali file baru bernama Latihan-1.js di dalam folder praktikum-5/. Ketik “touch latihan-1.js”.

```
HP@LAPTOP-PK3JNVV7 MINGW64 /c/2025573010117_Praktikum-Algoritma_dan-Struktur_data/praktikum-5 (main)
$ touch latihan-1.js
```

15. Lanjut Buka file Latihan-1.js di VS Code, lalu ketikkan kode berikut.

```javascript
// Latihan 1: Identifikasi Kompleksitas

// Fungsi A
// Big O: O(1)
// Alasan: Operasi konstan, tidak bergantung pada n
function fnA(n) {
  return n * 2;
}

// Fungsi B
// Big O: O(n^2)
// Alasan: Dua perulangan bersarang (nested loop)
function fnB(n) {
  for (let i = 0; i < n; i++) {
    for (let j = 0; j < n; j++) {
      // console.log(i, j);
    }
  }
}

// Fungsi C
// Big O: O(log n)
// Alasan: i dikali 2 setiap iterasi (pertumbuhan eksponensial)
function fnC(n) {
  for (let i = 1; i < n; i *= 2) {
    // console.log(i);
  }
}

// Fungsi D
// Big O: O(n^3)
// Alasan: Tiga perulangan bersarang
function fnD(n) {
  let arr = Array.from({ length: n }, (_, i) => i);

  arr.forEach((x) => {
    arr.forEach((y) => {
      arr.forEach((z) => {
        // console.log(x, y, z);
      });
    });
  });
}

// Fungsi untuk menghitung waktu eksekusi
function hitungKompleksitas(n, fn) {
  const start = Date.now();
  fn(n);
  const end = Date.now();
  console.log(fn.name + " => " + (end - start) + " ms");
}

// Pengujian
const n = 1000;

hitungKompleksitas(n, fnA);
hitungKompleksitas(n, fnB);
hitungKompleksitas(n, fnC);

// Hati-hati: fnD sangat berat jika n besar
hitungKompleksitas(50, fnD);
```

16. Simpan file jalankan diterminal ketik “node latihan-1.js” lalu amati hasilnya.

```
HP@LAPTOP-PK3JNVV7 MINGW64 /c/2025573010117_Praktikum-Algoritma_dan-Struktur_data/praktikum-5 (main)
$ node latihan-1.js
fnA => 0 ms
fnB => 5 ms
fnC => 0 ms
fnD => 1 ms
```

17. Setelah itu langkah selanjutnya yaitu membuat folder tugas dan 2 file tugas didalamnya dengan cara ketik “mkdir tugas” “touch tugas/tugas-1.js tugas/tugas-2.js”.

```
HP@LAPTOP-PK3JNVV7 MINGW64 /c/2025573010117_Praktikum-Algoritma_dan-Struktur_data/praktikum-5 (main)
$ mkdir tugas

HP@LAPTOP-PK3JNVV7 MINGW64 /c/2025573010117_Praktikum-Algoritma_dan-Struktur_data/praktikum-5 (main)
$ touch tugas/tugas-1.js tugas/tugas-2.js
```

18. Buka file tugas-1.js di VS Code, lalu ketikkan kode berikut.

```javascript
// ================================
// TUGAS 1 - Analisis & Refactor
// ================================

// ======================================
// FUNGSI A - INTERSECTION ARRAY
// ======================================

// Versi O(n^2)
function intersectionN2(arr1, arr2) {
  // Time: O(n^2), Space: O(1)
  const result = [];
  for (let i = 0; i < arr1.length; i++) {
    for (let j = 0; j < arr2.length; j++) {
      if (arr1[i] === arr2[j] && !result.includes(arr1[i])) {
        result.push(arr1[i]);
      }
    }
  }
  return result;
}

// Versi O(n) menggunakan Set
function intersectionN(arr1, arr2) {
  // Time: O(n), Space: O(n)
  const set2 = new Set(arr2);
  const result = [];

  for (let item of arr1) {
    if (set2.has(item)) {
      result.push(item);
    }
  }

  return [...new Set(result)]; // hapus duplikat
}

// ======================================
// FUNGSI B - KELOMPOK ANAGRAM
// ======================================

function groupAnagrams(words) {
  // Time: O(n log k), Space: O(n)
  // n = jumlah kata, k = panjang kata

  const map = new Map();

  for (let word of words) {
    const key = word.split("").sort().join("");

    if (!map.has(key)) {
      map.set(key, []);
    }

    map.get(key).push(word);
  }

  return Array.from(map.values());
}

// ======================================
// FUNGSI C - SUM = KUADRAT
// ======================================

// Versi O(n^3)
function sumSquareN3(arr) {
  // Time: O(n^3), Space: O(1)
  const n = arr.length;

  for (let i = 0; i < n; i++) {
    for (let j = 0; j < n; j++) {
      for (let k = 0; k < n; k++) {
        if (arr[i] + arr[j] === arr[k] * arr[k]) {
          return true;
        }
      }
    }
  }
  return false;
}

// Versi O(n log n)
function sumSquareOptimized(arr) {
  // Time: O(n log n), Space: O(1)
  arr.sort((a, b) => a - b);

  const n = arr.length;

  for (let k = n - 1; k >= 0; k--) {
    let left = 0;
    let right = k - 1;

    const target = arr[k] * arr[k];

    while (left <= right) {
      const sum = arr[left] + arr[right];

      if (sum === target) {
        return true;
      } else if (sum < target) {
        left++;
      } else {
        right--;
      }
    }
  }

  return false;
}

// ======================================
// TEST & PENGUKURAN WAKTU
// ======================================

// Data besar
const size = 5000;
const arr1 = Array.from({ length: size }, () =>
  Math.floor(Math.random() * 10000),
);
const arr2 = Array.from({ length: size }, () =>
  Math.floor(Math.random() * 10000),
);

const words = ["eat", "tea", "tan", "ate", "nat", "bat"];

// Fungsi A test
console.time("Intersection O(n^2)");
intersectionN2(arr1, arr2);
console.timeEnd("Intersection O(n^2)");

console.time("Intersection O(n)");
intersectionN(arr1, arr2);
console.timeEnd("Intersection O(n)");

// Fungsi B test
console.log("Anagram:", groupAnagrams(words));

// Fungsi C test
console.time("SumSquare O(n^3)");
sumSquareN3([1, 2, 3, 4, 5, 6, 7, 8, 9]);
console.timeEnd("SumSquare O(n^3)");

console.time("SumSquare O(n log n)");
sumSquareOptimized([1, 2, 3, 4, 5, 6, 7, 8, 9]);
console.timeEnd("SumSquare O(n log n)");
```

19. Kemudian Simpan file jalankan diterminal ketik “node tugas-1.js” lalu amati hasilnya.

```
HP@LAPTOP-PK3JNVV7 MINGW64 /c/2025573010117_Praktikum-Algoritma_dan-Struktur_data/praktikum-5/tugas (main)
$ node tugas-1.js
Intersection O(n^2): 76.822ms
Intersection O(n): 3.375ms
Anagram: [ [ 'eat', 'tea', 'ate' ], [ 'tan', 'nat' ], [ 'bat' ] ]
SumSquare O(n^3): 0.171ms
SumSquare O(n log n): 0.203ms
```

20. Lanjut Buka file tugas-2.js di VS Code, lalu ketikkan kode berikut.

```javascript
// ========================================
// TUGAS 2 - VISUALISASI PERTUMBUHAN BIG O
// ========================================

// ===========================
// DEFINISI FUNGSI
// ===========================

// O(1)
function fn_O1(n) {
  // Time: O(1), space: O(1)
  return n + 1;
}

// O(n)
function fn_On(n) {
  // Time: O(n), Space: O(1)
  let sum = 0;
  for (let i = 0; i < n; i++) {
    sum += i;
  }
  return sum;
}

// O(n log n)
function fn_OnLogn(n) {
  // Time: O(n log n), Space: O(1)
  let count = 0;
  const logN = Math.floor(Math.log2(n));

  for (let i = 0; i < n; i++) {
    for (let j = 0; j < logN; j++) {
      count++;
    }
  }
  return count;
}

// O(n^2)
function fn_On2(n) {
  // Time: O(n^2), Space: O(1)
  let count = 0;

  for (let i = 0; i < n; i++) {
    for (let j = 0; j < n; j++) {
      count++;
    }
  }
  return count;
}

// ===========================
// BENCHMARK FUNCTION
// ===========================

function benchmarkSemua(ukuranData) {
  console.log("===== HASIL BENCHMARK =====");

  for (let n of ukuranData) {
    console.log(`\nUkuran data: ${n}`);

    console.time("O(1)");
    fn_O1(n);
    console.timeEnd("O(1)");

    console.time("O(n)");
    fn_On(n);
    console.timeEnd("O(n)");

    console.time("O(n log n)");
    fn_OnLogn(n);
    console.timeEnd("O(n log n)");

    console.time("O(n^2)");
    fn_On2(n);
    console.timeEnd("O(n^2)");
  }
}

// ===========================
// JALANKAN BENCHMARK
// ===========================

benchmarkSemua([100, 500, 1000, 5000, 10000]);

// ===========================
// ANALISIS HASIL (KOMENTAR)
// ===========================

/*
HASIL PENGAMATAN (umumnya):

1. O(1):
   - Waktu hampir sama untuk semua ukuran data
   - Tidak terpengaruh oleh besar kecilnya n

2. O(n):
   - Waktu bertambah secara linear
   - Jika n naik 2x, waktu kira-kira juga naik 2x

3. O(n log n):
   - Lebih cepat dari O(n^2), tapi lebih lambat dari O(n)
   - Pertumbuhan mulai terasa saat n besar

4. O(n^2):
   - Waktu meningkat sangat drastis
   - Jika n naik 10x, waktu bisa naik ~100x
   - Paling lambat di antara semua

KESIMPULAN:
- Pola pertumbuhan waktu eksekusi konsisten dengan teori Big O
- Semakin tinggi kompleksitas, semakin cepat waktu eksekusi meningkat
- O(1) paling efisien, O(n^2) paling tidak efisien untuk data besar
*/
```

21. Simpan file jalankan diterminal ketik “node tugas-2.js” lalu amati hasilnya.

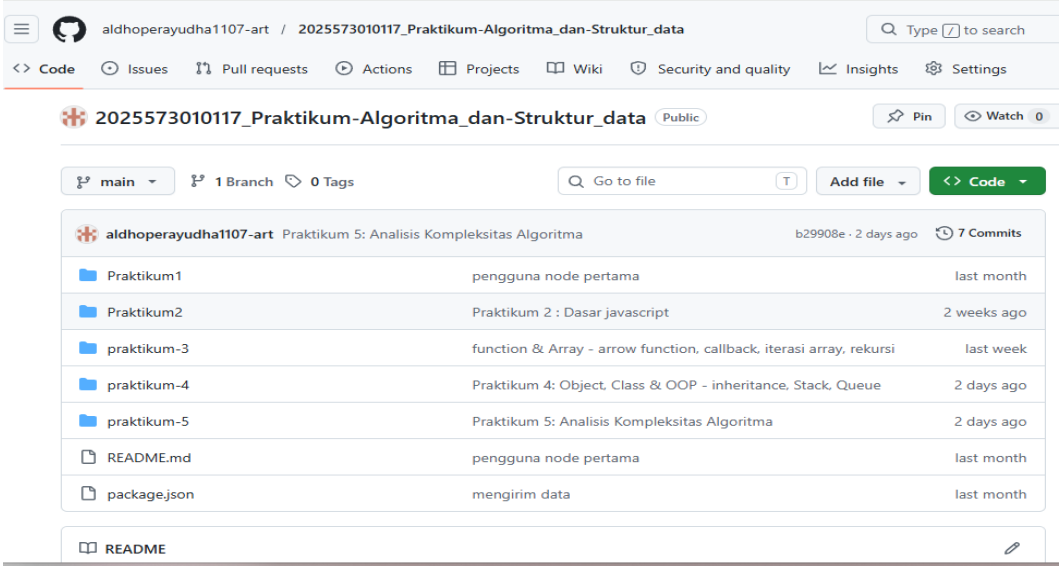
```
HP@LAPTOP-PK3JNVV7 MINGW64 /c/2025573010117_Praktikum-Algoritma_dan-Struktur_data/praktikum-5/tugas (main)
$ node tugas-2.js
===== HASIL BENCHMARK =====

Ukuran data: 100
O(1): 0.041ms
O(n): 0.067ms
O(n log n): 0.146ms
O(n^2): 0.436ms

Ukuran data: 500
O(1): 0.068ms
O(n): 0.038ms
O(n log n): 0.179ms
O(n^2): 1.225ms

Ukuran data: 1000
O(1): 0.009ms
O(n): 0.086ms
O(n log n): 0.117ms
O(n^2): 3.683ms

Ukuran data: 5000
O(1): 0.011ms
O(n): 0.019ms
O(n log n): 0.301ms
O(n^2): 74.993ms

Ukuran data: 10000
O(1): 0.015ms
O(n): 0.055ms
O(n log n): 1.478ms
O(n^2): 296.481ms
```

22. Jika sudah menyelesaikan semua langkah maka dilanjutkan push ke akun Github. Dengan cara ketik “git status” untuk mengecek “git add praktikum-5/” “git commit -m "Praktikum 5: Analisis Kompleksitas Algoritma"” “git push origin main”.

```
HP@LAPTOP-PK3JNVV7 MINGW64 /c/2025573010117_Praktikum-Algoritma_dan-Struktur_data/praktikum-5 (main)
$ cd ..

HP@LAPTOP-PK3JNVV7 MINGW64 /c/2025573010117_Praktikum-Algoritma_dan-Struktur_data (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add/rm <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        deleted:    Praktikum2/praktikum-2/package.json
        deleted:    package.json
        modified:   praktikum-3/02-arrow-callback.js

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        Praktikum1/package.json
        praktikum-5/

no changes added to commit (use "git add" and/or "git commit -a")

HP@LAPTOP-PK3JNVV7 MINGW64 /c/2025573010117_Praktikum-Algoritma_dan-Struktur_data (main)
$ git add praktikum-5/
warning: in the working copy of 'praktikum-5/package.json', LF will be replaced by CRLF the next time Git touches it
```

```
HP@LAPTOP-PK3JNVV7 MINGW64 /c/2025573010117_Praktikum-Algoritma_dan-Struktur_data (main)
$ git commit -m "Praktikum 5: Analisis Kompleksitas Algoritma"
[main b29908e] Praktikum 5: Analisis Kompleksitas Algoritma
 8 files changed, 648 insertions(+)
 create mode 100644 praktikum-5/01-intro-kompleksitas.js
 create mode 100644 praktikum-5/02-kelas-big-o.js
 create mode 100644 praktikum-5/03-space-complexity.js
 create mode 100644 praktikum-5/04-refactor.js
 create mode 100644 praktikum-5/latihan-1.js
 create mode 100644 praktikum-5/package.json
 create mode 100644 praktikum-5/tugas/tugas-1.js
 create mode 100644 praktikum-5/tugas/tugas-2.js

HP@LAPTOP-PK3JNVV7 MINGW64 /c/2025573010117_Praktikum-Algoritma_dan-Struktur_data (main)
$ git push origin main
Enumerating objects: 13, done.
Counting objects: 100% (13/13), done.
Delta compression using up to 12 threads
Compressing objects: 100% (12/12), done.
Writing objects: 100% (12/12), 6.66 KiB | 681.00 KiB/s, done.
Total 12 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/aldhoperayudha1107-art/2025573010117_Praktikum-Algoritma_dan-Struktur_data.git
   8905638..b29908e  main -> main
```

23. Setelah di push ke Github, Cek apakah datanya sudah di update jika sudah maka tampilannya seperti berikut.



BAB IV
PENUTUP

1. Kesimpulan

Berdasarkan hasil praktikum yang telah dilakukan, dapat disimpulkan bahwa analisis kompleksitas algoritma merupakan aspek penting dalam pengembangan program yang efisien. Dengan memahami Big O Notation, kita dapat mengetahui bagaimana performa suatu algoritma ketika ukuran data semakin besar.

Konsep time complexity membantu dalam mengukur kecepatan eksekusi algoritma, sedangkan space complexity berkaitan dengan penggunaan memori selama program berjalan. Dari percobaan yang dilakukan, terlihat bahwa algoritma dengan kompleksitas lebih rendah seperti O(1) dan O(n) memiliki kinerja yang jauh lebih baik dibandingkan algoritma dengan kompleksitas tinggi seperti $O(n^2)$ atau $O(2^n)$.

Selain itu, penggunaan teknik optimasi seperti refactoring dan pemanfaatan struktur data yang tepat (misalnya Set atau Object) dapat meningkatkan efisiensi algoritma secara signifikan. Oleh karena itu, pemahaman terhadap pola-pola kompleksitas sangat diperlukan dalam menulis kode yang optimal.

2. Saran

Adapun saran yang dapat diberikan adalah :

- Mahasiswa disarankan untuk lebih sering berlatih menganalisis kompleksitas algoritma pada berbagai kasus
- Menguji performa program dengan data berukuran besar agar lebih memahami perbedaan tiap kompleksitas
- Mempelajari lebih dalam teknik optimasi algoritma dan penggunaan struktur data yang tepat
- Praktikum selanjutnya diharapkan memberikan studi kasus yang lebih kompleks agar kemampuan analisis semakin berkembang